

Command Pattern

Wissenschaftliche Themenarbeit

Henning Franzen, Jan Osing, Jonas Schierhold

Januar 2026



Matrikelnummern:	202326004, 202326072, 202326008
Studiengang:	Duales Studium Bachelor of Arts Wirtschaftsinformatik
Gruppe:	IT-BW-18
Themensteller:	Prof. Dr. Priemer
Abgabedatum:	20. Juli 2026

Inhaltsverzeichnis

1	Einleitung	1
1.1	Überblick	1
2	Klassifizierung anhand des Vier-Punkte-Schemas	2
2.1	Name	2
2.2	Problemstellung	2
2.3	Lösung	3
2.4	Konsequenzen	4
3	Erweiterungsmöglichkeiten	6
3.1	Undo- und Redo-Mechanismen	6
3.2	Makros (Befehlsketten)	7
3.3	Warteschlangen und Logging	7
4	Abgrenzung und moderne Alternativen	8
5	Praxisbeispiel in Java	9
6	Fazit	12

Abkürzungsverzeichnis

UI	User Interface (Benutzeroberfläche)
UML	Unified Modeling Language

1 Einleitung

1.1 Überblick

Die Entwicklung moderner Softwaresysteme erfordert Lösungen, die sowohl flexibel als auch wartbar und erweiterbar sind. Entwurfsmuster (Design Patterns) stellen hierfür bewährte Konzepte bereit, indem sie wiederkehrende Probleme der objektorientierten Softwareentwicklung auf standardisierte Weise lösen. Sie bieten keine fertigen Implementierungen, sondern beschreiben allgemeine Lösungsansätze, die abhängig vom jeweiligen Anwendungsfall angepasst werden können.

Das Command Pattern gehört zu den verhaltensorientierten Entwurfsmustern und dient dazu, Anweisungen oder Operationen als eigenständige Objekte zu kapseln. Durch diese Kapselung werden aufrufende und ausführende Komponenten voneinander entkoppelt, wodurch sich Befehle flexibel verwalten, weitergeben oder zu einem späteren Zeitpunkt ausführen lassen. Aufgrund dieser Eigenschaften findet das Muster in verschiedenen Bereichen der Softwareentwicklung Anwendung, etwa bei grafischen Benutzeroberflächen, Undo/Redo-Funktionalität oder der Verarbeitung von Befehlswarteschlangen.¹

Ziel dieser Hausarbeit ist es, das Command Pattern systematisch zu analysieren und dessen Funktionsweise sowie Einsatzmöglichkeiten zu erläutern. Zunächst wird das Entwurfsmuster anhand des Vier-Punkte-Schemas mit den Aspekten *Name*, *Problem*, *Lösung* und *Konsequenzen* vorgestellt. Anschließend werden einige Erweiterungsszenarien erklärt und mithilfe von UML-Diagrammen tiefer veranschaulicht. Danach wird die praktische Umsetzung des Command Patterns anhand einer beispielhaften Implementierung erläutert, bevor mit einem kurzen Fazit abgeschlossen wird.

¹Schiele, 2026

2 Klassifizierung anhand des Vier-Punkte-Schemas

2.1 Name

Das Entwurfsmuster trägt den Namen Command-Pattern. Im deutschsprachigen Raum wird es als Kommando-Muster bezeichnet. Es gehört zur Kategorie der Verhaltensmuster (Behavioral Patterns).

2.2 Problemstellung

In vielen Anwendungen werden Benutzeraktionen direkt mit der auszuführenden Programmlogik verknüpft. Ein typisches Beispiel ist eine grafische Benutzeroberfläche (UI), in der verschiedene Buttons bestimmte Funktionen auslösen. So kann beispielsweise ein Button zum Speichern eines Dokuments, ein weiterer zum Drucken und ein dritter zum Löschen einer Datei dienen. Wird die jeweilige Funktion direkt im Code des Buttons implementiert, entsteht eine enge Kopplung zwischen der Benutzeroberfläche und der eigentlichen Geschäftslogik.

Diese enge Abhängigkeit führt zu mehreren Problemen. Zum einen wird die Wiederverwendbarkeit der Funktionalität eingeschränkt, da dieselbe Aktion beispielsweise nicht ohne Weiteres über Tastenkombinationen, Menüeinträge oder andere Eingabemethoden ausgelöst werden kann. Zum anderen erschwert die direkte Verknüpfung die Wartung und Erweiterung der Software. Jede Änderung an einer Funktion erfordert häufig auch Anpassungen an den entsprechenden UI-Komponenten. Mit zunehmender Anzahl an Buttons und Funktionen wird der Quellcode dadurch unübersichtlicher und schwieriger zu pflegen.

Ein weiteres Problem ergibt sich, wenn zusätzliche Anforderungen wie eine Undo-/Redo-Funktion, das Protokollieren ausgeführter Aktionen oder die zeitversetzte Ausführung von Befehlen umgesetzt werden sollen. Sind die Aktionen fest in den UI-Elementen implementiert, lassen sich solche Erweiterungen nur mit erheblichem Aufwand realisieren.

Zur Veranschaulichung kann eine Textverarbeitungsanwendung betrachtet werden. Dort besitzt die Benutzeroberfläche unter anderem die Buttons *Speichern*, *Drucken* und *Rückgängig*. Jeder Button soll eine bestimmte Aktion auslösen, unabhängig davon, ob diese über einen Mausklick, einen Menüeintrag oder eine Tastenkombination aktiviert wird. Um diese Flexibilität zu erreichen, muss die Benutzeroberfläche von der eigentlichen Ausführung der Aktionen entkoppelt werden. Genau an dieser Stelle setzt das Command Pattern an, indem jede Benutzeraktion als eigenständiges Befehlsobjekt gekapselt wird. Die UI kennt lediglich das jeweilige Kommando und muss keine Details über dessen konkrete Implementierung oder Ausführung besitzen.²

2.3 Lösung

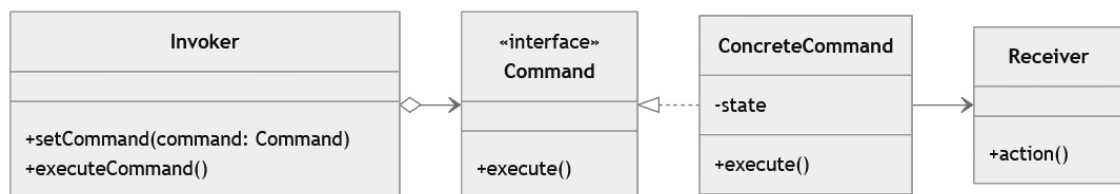


Abbildung 1: UML Klassendiagramm: Command Pattern

Das Command-Pattern löst die enge Kopplung zwischen Sender und Empfänger durch eine objektorientierte Kapselung. Es transformiert eine Anfrage oder Operation in ein eigenständiges Objekt, das alle für die Ausführung notwendigen Informationen in sich speichert. Dadurch wird der direkte Methodenaufruf abstrahiert. Die Architektur nutzt hierfür ein gemeinsames Interface, welches eine einzige Methode namens `execute()` vorschreibt. Der Sender der Anfrage interagiert ab sofort ausschließlich mit diesem Interface, wodurch er die konkrete Implementierung der dahinterliegenden Logik nicht mehr kennen muss.³

Die Funktionsweise des Musters basiert auf dem strukturierten Zusammenspiel von vier zentralen Akteuren.

²Refactoring.Guru, 2026

³Gamma et al., 1995

- **Klient (Client):** Der Klient steuert den initialen Ablauf. Er erzeugt das konkrete Kommando-Objekt und injiziert dabei den passenden Empfänger. Anschließend weist er das fertig konfigurierte Kommando dem Aufrufer zu.
- **Aufrufer (Invoker):** Der Aufrufer fungiert als reiner Auslöser der Anfrage und hält eine Referenz auf das Command-Interface. Da er die konkrete Implementierung nicht kennt, ist ihm die genaue Funktion des Kommandos verborgen. Zu einem festgelegten Zeitpunkt ruft er lediglich die `execute()`-Methode des Kommandos auf.
- **Kommando (Command):** Das Kommando bildet das Bindeglied der Architektur und implementiert das gemeinsame Interface. Es kapselt die Anfrage und delegiert in seiner `execute()`-Methode die eigentliche Arbeit an den Empfänger. Das Kommando führt die komplexe Logik folglich fast nie selbst aus.
- **Empfänger (Receiver):** Der Empfänger enthält die eigentliche Geschäftslogik und führt die tatsächlichen Berechnungen oder Zustandsänderungen durch. Er kommuniziert niemals direkt mit dem Aufrufer, sondern erhält seine Arbeitsaufträge ausschließlich über das zwischengeschaltete Kommando-Objekt.

Durch diese klare Aufteilung wird eine vollständige Entkopplung der Komponenten erreicht: Der Aufrufer erteilt lediglich den Befehl, das Kommando transportiert sowie übersetzt ihn, und der Empfänger führt ihn schließlich aus. Nachträgliche Änderungen an der Geschäftslogik des Empfängers bleiben somit ohne jegliche Auswirkung auf den Aufrufer.

2.4 Konsequenzen

Dieses Vorgehen bringt deutliche Vor- und Nachteile mit sich. Der größte Vorteil ist die starke Entkopplung der Komponenten: Absender und Empfänger sind völlig unabhängig, sodass neue Kommandos jederzeit ergänzt werden können, ohne den

bestehenden Code zu verändern. Da jeder Befehl ein eigenständiges Objekt ist, lässt er sich zudem isoliert testen. Der primäre Nachteil ist die steigende Komplexität, da für jeden Befehl eine eigene Klasse angelegt werden muss und die Anzahl der Klassen im System dadurch stark wächst. Hinzu kommt konkreter Zusatzaufwand in den Erweiterungen: Undo-/Redo-Stacks halten vergangene Zustände im Speicher vor, und bei der nebenläufigen Abarbeitung von Befehlswarteschlangen müssen Fragen der Thread-Sicherheit beachtet werden. Ein wesentlicher Vorteil bleibt jedoch, dass diese Struktur überhaupt erst solche mächtigen funktionalen Erweiterungen ermöglicht.

3 Erweiterungsmöglichkeiten

Das Command-Pattern bietet durch seine entkoppelte Struktur eine ideale Basis für fortgeschrittene Architekturen.

3.1 Undo- und Redo-Mechanismen

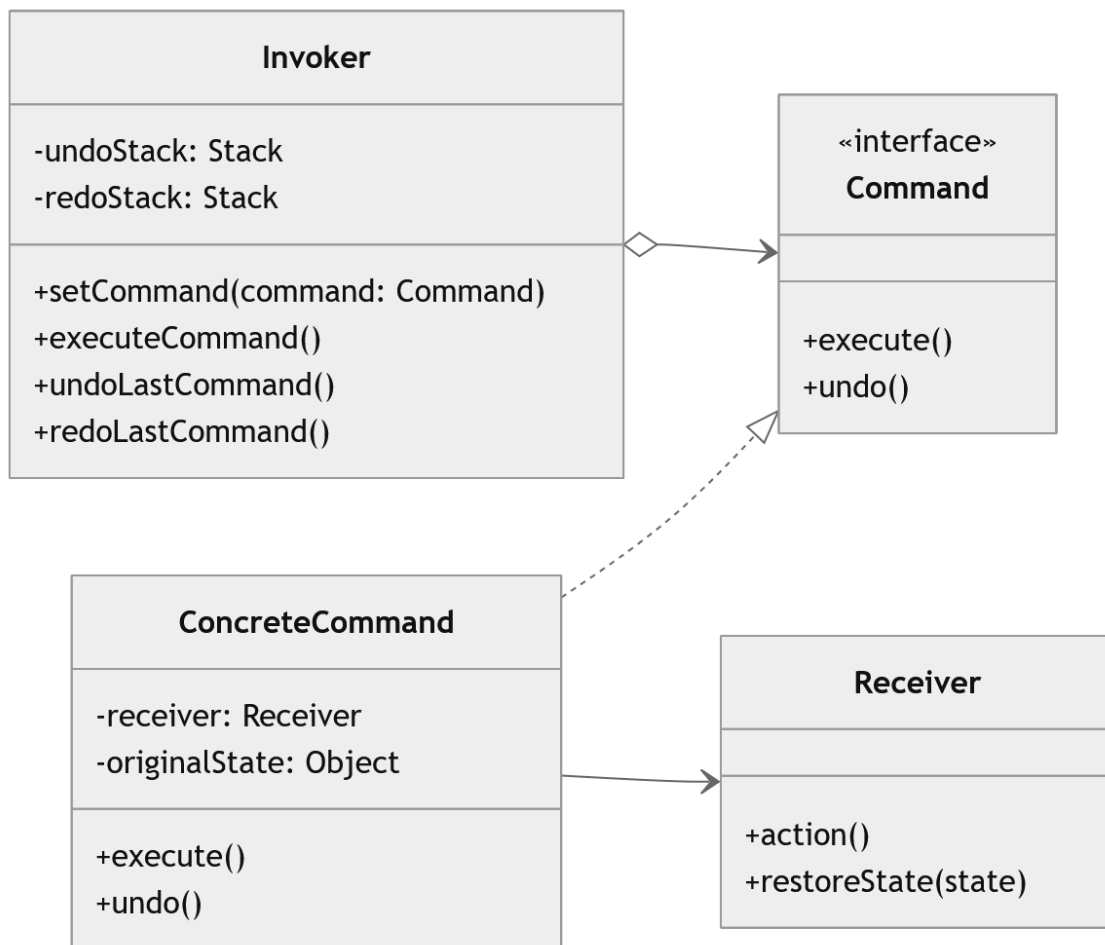


Abbildung 2: UML Klassendiagramm: Command Pattern Erweiterung mit Undo/Redo

Aktionen können leicht rückgängig gemacht werden. Dafür wird das Interface um eine `undo()`-Methode ergänzt. Das Kommando speichert intern den Zustand vor der Ausführung. Bei einem Undo-Aufruf stellt es diesen Ursprungszustand wieder her.

Redo-Operationen können das Kommando danach einfach erneut ausführen. Um einen komplexen Zustand wiederherzustellen, lässt sich das Command-Pattern mit dem Memento-Pattern⁴ kombinieren.

3.2 Makros (Befehlsketten)

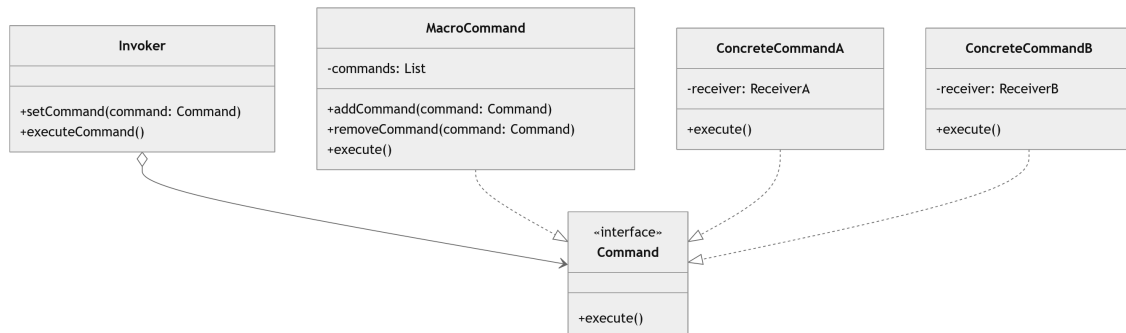


Abbildung 3: UML Klassendiagramm: Command Pattern Erweiterung mit Makros

Mehrere Einzelkommandos können zu sogenannten Makros gebündelt werden. Dies geschieht über ein übergeordnetes Makro-Kommando. Es implementiert ebenfalls das Standard-Interface. In seiner `execute()`-Methode arbeitet es nacheinander eine Liste von Teilkommandos ab.

3.3 Warteschlangen und Logging

Befehle lassen sich in einer Warteschlange (Queue) speichern. Dadurch können sie asynchron zu einem späteren Zeitpunkt ausgeführt werden, was bei der Verwaltung von Thread-Pools von großen Vorteil ist. Zudem ermöglicht die reine Objektform ein einfaches Logging. Jeder ausgeführte Befehl kann in einer Datei gespeichert oder in der Konsole ausgegeben werden. Bei einem Systemabsturz lassen sich die Aktionen aus dem Log wiederherstellen.

⁴Weiterführende Info: Memento-Pattern (letzter Aufruf: 19.07.2026)

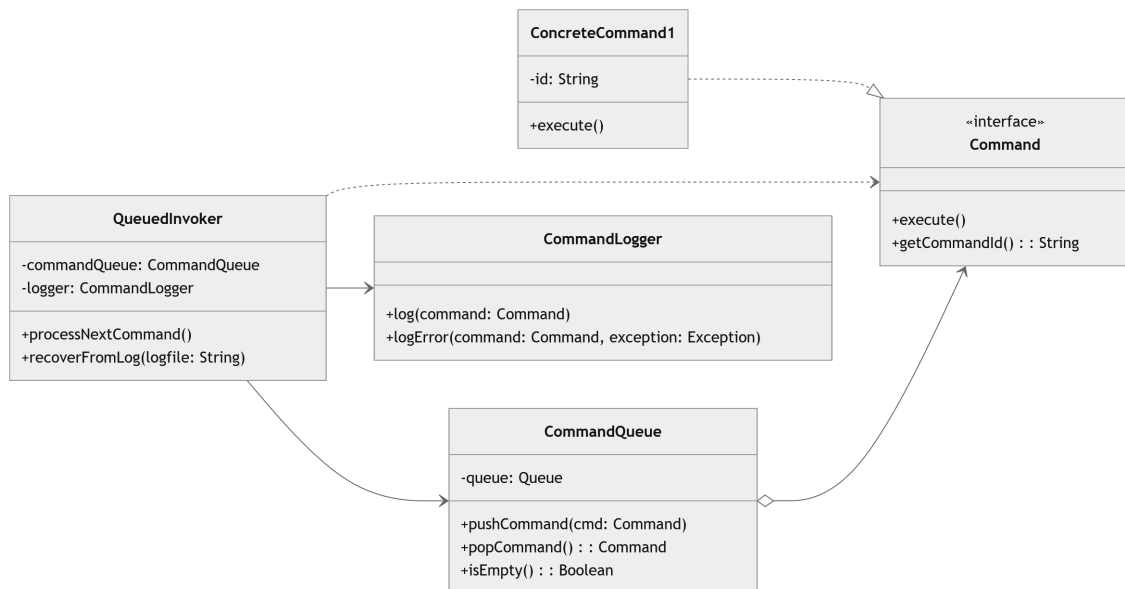


Abbildung 4: UML Klassendiagramm: Command Pattern Erweiterung durch Queue und Logging

4 Abgrenzung und moderne Alternativen

Strukturell ähnelt das Command-Pattern dem Strategy-Pattern: Beide kapseln Verhalten hinter einem Interface. Die Intention unterscheidet sich jedoch. Das Strategy-Pattern kapselt *austauschbare Algorithmen* für ein und dieselbe Aufgabe, während das Command-Pattern eine *vollständige Anfrage* inklusive Empfänger und Parametern als Objekt kapselt, um sie zu speichern, weiterzugeben oder rückgängig zu machen.⁵

In modernen Java-Versionen (ab Java 8) lassen sich einfache Kommandos ohne Zustand auch als Lambda-Ausdruck formulieren, da das Command-Interface ein funktionales Interface ist. Dies reduziert den in den Konsequenzen genannten Klassenwildwuchs. Sobald ein Kommando jedoch zusätzliche Funktionen wie `undo()` oder internen Zustand benötigt, bleibt die klassenbasierte Umsetzung überlegen.

⁵Gamma et al., 1995

5 Praxisbeispiel in Java

Zur Veranschaulichung wird das Command-Pattern nachfolgend anhand eines kompakten, aber vollständigen Beispiels in Java umgesetzt. Als Szenario dient eine Fernbedienung, die eine Lampe ein- und ausschaltet. Das Beispiel deckt alle vier zuvor beschriebenen Rollen des Musters ab (vgl. Abbildung 1) und greift zusätzlich die in Abschnitt 2 vorgestellte Undo-Funktion auf:

- **Kommando (Command):** das Interface `Command`,
- **Empfänger (Receiver):** die Klasse `Light`,
- **Konkrete Kommandos:** `LightOnCommand` und `LightOffCommand`,
- **Aufrufer (Invoker):** die Klasse `RemoteControl`,
- **Klient (Client):** die Klasse `Main`.

Den Ausgangspunkt bildet das gemeinsame Interface `Command`. Es schreibt mit `execute()` das Auslösen und mit `undo()` das Rückgängigmachen eines Befehls vor und stellt damit die einheitliche Schnittstelle dar, über die sämtliche Kommandos angesprochen werden.

```
1 // Kommando-Interface
2 public interface Command {
3     void execute();
4     void undo();
5 }
6
```

Listing 1: Das Kommando-Interface `Command`

Der Empfänger enthält die eigentliche Geschäftslogik. Im Beispiel ist dies die Klasse `Light`, welche die konkreten Operationen zum Ein- und Ausschalten bereitstellt. Sie besitzt keinerlei Kenntnis vom Command-Pattern und könnte unverändert auch ohne dieses verwendet werden.

```

1 // Empfänger - führt die eigentliche Arbeit aus
2 public class Light {
3     public void on() { System.out.println("Light is ON"); }
4     public void off() { System.out.println("Light is OFF"); }
5 }
6

```

Listing 2: Der Empfänger Light

Die konkreten Kommandos kapseln jeweils eine Anfrage an den Empfänger. Sie implementieren das Interface `Command` und halten eine Referenz auf das `Light`-Objekt. In `execute()` delegieren sie die Arbeit an den Empfänger, während `undo()` die jeweils gegenteilige Operation auslöst.

```

1 // Konkretes Kommando - kapselt "Licht einschalten" als Objekt
2 public class LightOnCommand implements Command {
3     private final Light light;
4
5     public LightOnCommand(Light light) { this.light = light; }
6
7     public void execute() { light.on(); }
8     public void undo() { light.off(); }
9 }
10
11 // Konkretes Kommando - kapselt "Licht ausschalten" als Objekt
12 public class LightOffCommand implements Command {
13     private final Light light;
14
15     public LightOffCommand(Light light) { this.light = light; }
16
17     public void execute() { light.off(); }
18     public void undo() { light.on(); }
19 }
20

```

Listing 3: Die konkreten Kommandos `LightOnCommand` und `LightOffCommand`

Der Aufrufer löst Befehle aus, ohne deren konkrete Implementierung zu kennen.

Die Klasse `RemoteControl` hält eine Referenz auf das `Command`-Interface und ruft bei einem Tastendruck dessen `execute()`-Methode auf. Zusätzlich merkt sie sich den zuletzt ausgeführten Befehl, sodass dieser über `pressUndo()` rückgängig gemacht werden kann. Welches Kommando dabei ausgeführt wird und was es bewirkt, bleibt dem Aufrufer verborgen.

```
1 // Aufrufer - löst Kommandos aus, ohne ihre Wirkung zu kennen
2 public class RemoteControl {
3     private Command command;
4     private Command lastCommand;
5
6     public void setCommand(Command c) { command = c; }
7     public void pressButton() { command.execute(); lastCommand =
    command; }
8     public void pressUndo()    { lastCommand.undo(); }
9 }
10
```

Listing 4: Der Aufrufer `RemoteControl`

Der Klient verknüpft schließlich die Komponenten miteinander. Er erzeugt den Empfänger, instanziiert die konkreten Kommandos mit diesem Empfänger und übergibt sie dem Aufrufer. Nach dem Ein- und Ausschalten macht ein abschließender Aufruf von `pressUndo()` den zuletzt ausgeführten Befehl rückgängig und schaltet die Lampe wieder ein.

```
1 public class Main {
2     public static void main(String[] args) {
3         Light light = new Light();
4         RemoteControl remote = new RemoteControl();
5
6         remote.setCommand(new LightOnCommand(light));
7         remote.pressButton();
8
9         remote.setCommand(new LightOffCommand(light));
10        remote.pressButton();
11
12        remote.pressUndo();
13    }
14 }
```

```
13     }  
14 }  
15
```

Listing 5: Der Klient Main

Die Ausführung des Programms erzeugt die folgende Ausgabe:

```
1 Light is ON  
2 Light is OFF  
3 Light is ON  
4
```

Listing 6: Konsolenausgabe des Beispiels

Das Beispiel verdeutlicht die zentrale Eigenschaft des Musters: Die Klasse `RemoteControl` besitzt keinerlei Wissen über die Klasse `Light`. Der Aufrufer kommuniziert ausschließlich mit dem `Command`-Interface, während die eigentliche Logik im Empfänger gekapselt bleibt. Soll die Fernbedienung künftig ein anderes Gerät steuern, muss lediglich ein neues konkretes Kommando ergänzt werden. Weder der Aufrufer noch der bestehende Code müssen dafür verändert werden, was die im vorherigen Abschnitt beschriebene Entkopplung praktisch belegt. Dasselbe Prinzip findet sich in etablierten Frameworks wieder, etwa in `javax.swing.Action`, `QAction` in Qt oder den Actions in Redux.

6 Fazit

Das Command Pattern ist ein bewährtes Entwurfsmuster zur Entkopplung von aufrufenden und ausführenden Komponenten. Durch die Kapselung von Befehlen als eigenständige Objekte erhöht es die Flexibilität, Wartbarkeit und Erweiterbarkeit von Softwaresystemen. Gleichzeitig bildet es die Grundlage für weiterführende Funktionen wie Undo-/Redo-Mechanismen, Makros oder die Verarbeitung von Befehlswarteschlangen.

Die Analyse anhand des Vier-Punkte-Schemas hat die Funktionsweise sowie die Vor- und Nachteile des Musters verdeutlicht. Das anschließende Java-Beispiel zeigte, wie die theoretischen Konzepte in einer praktischen Implementierung umgesetzt werden können. Insgesamt eignet sich das Command Pattern insbesondere für Anwendungen, in denen Befehle flexibel verwaltet, erweitert oder zeitlich unabhängig ausgeführt werden sollen. Trotz des erhöhten Implementierungsaufwands überwiegen in vielen Anwendungsfällen die Vorteile einer klar strukturierten und entkoppelten Softwarearchitektur. Nach eigener Einschätzung sollte das Muster jedoch bewusst eingesetzt werden: Sein voller Nutzen entfaltet sich erst, wenn Anforderungen wie Undo/Redo, Makros oder Warteschlangen tatsächlich bestehen. Für einfache, einmalige Aktionen stellt der resultierende Klassenaufwand hingegen eine Überkonstruktion dar, die sich mit funktionalen Ansätzen schlanker lösen lässt.

Literatur

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Gips, C. (n. d.). *Command-Pattern*. Hochschule Bielefeld (HSBI). Verfügbar 16. Juli 2026 unter https://www.hsbi.de/elearning/data/FH-Bielefeld/lm_data/lm_1359639/pattern/command.html
- Johnson, E. G. (2023, Oktober). Design Patterns: Command [Aufgerufen am 19. Juli 2026]. https://medium.com/@EG_Johnson/design-patterns-command-c644e1b16f90
- Nystrom, R. (2014). *Command*. Verfügbar 16. Juli 2026 unter <https://gameprogrammingpatterns.com/command.html>
- Refactoring.Guru. (2026). *Command* [Abgerufen am 19. Juli 2026]. Verfügbar 19. Juli 2026 unter <https://refactoring.guru/design-patterns/command>
- Schiele, V. (2026). *Kommando-Entwurfsmuster* [Copyright © 2021–2024, Veit Schiele] (24.3.0). Python Basics. Verfügbar 7. Juli 2026 unter <https://python-basics-tutorial.readthedocs.io/de/24.3.0/oop/design/command.html>
- Shvets, A. (2026). *Command*. Refactoring.Guru. Verfügbar 16. Juli 2026 unter <https://refactoring.guru/design-patterns/command>